

PyQCU Strict MultiGrid 并行 Hopper 优化与最终对照

PyQCU 工程分析记录

2026 年 9 月 27 日

摘要

本轮针对快速迭代报告遗留的双 P100 通信热点，在 `cpp/cuda/qcu/src/apply_multigrid_strict.cu` 中实现了 Strict-MG vector-halo 的非阻塞交换与 local/remote 分区计算。active face 打包、pinned-host staging 和 `MPI_Isend/Irecv` 在独立交换流上执行，主流的 coarse local kernel 在收到 ghost 前即可运行；remote backward/forward correction 合并到单次 kernel。独立双向 MPI 真残差探针在 overlap off/on 下分别为 27/26 个外层迭代，真残差分别为 6.43×10^{-7} 和 7.52×10^{-7} 。

同一缓存、同一协议下，大格 c64 MG-2（双 P100， $\nu = 1$ ，coarse-tol = 0.3）采用两组反向顺序、各 5-solve 的配对测量；运行中位数的中位数由 4.654s 降至 4.337s，改善 6.81%。该 A/B 只隔离 overlap 效应，不作为最终协议性能结论。

最终 formal Cartesian 矩阵重新使用 c64 coarse-tol = 0.003、c128 coarse-tol = 3×10^{-5} 、reference 1 cold +2 warmup +5 steady，完成 72/72 个 exact-config comparison unit 和 144/144 个 side case。MG-2/3 共 48 组中 PyQCU 胜 38 组，中位比值 QUDA/PyQCU = 1.346；P100 为 1.158，V100 为 2.115。最大真残差分别低于 7.99×10^{-7} 和 7.97×10^{-7} 。overlap 仅默认用于 complex64；大格 c128 三层显式开启时会从 32 个外层迭代退化到 451 个，因此 complex128 默认保留原浮点累加顺序，并提供 fail-safe 环境开关。

1 目标、验收与边界

本轮目标是在不改变算子、容差、缓存资产或 MPI 拓扑契约的前提下，减少 Strict-MG coarse hopping 的通信阻塞。验收条件为：

1. 双 rank 独立 Python 真残差小于 5×10^{-6} ；
2. overlap off/on 的 c64 小格外迭代和真残差保持同等水平；
3. P100 大格 c64 MG-2 使用同缓存、同参数的两组反向顺序五次稳态回测；
4. c128 大格三层必须在默认路径恢复到原数值轨迹；
5. 全量矩阵、trace、reference 阶段记录继续通过 fail-closed 检查。

本文不把 trace 同步时间当作正式 solve-only 性能；正式 MG 加速比使用每个 trace 模式各自记录的 steady median，并要求两侧 config hash 和输入 bundle hash 完全一致。最终数据集不再用历史记录补位：所有大格单元都在 600 秒单元预算内完成，并保留 cold、两次 warmup、五次 steady 和每次求解的真残差记录。

2 实现

2.1 通信与计算重叠

`StrictVectorHalo<T>` 新增私有非阻塞交换流：

1. 在主流记录 input-ready event，交换流按 event 等待并打包 active face；
2. pinned-host 模式下完成 D2H staging 后一次同步，再按方向提交 `MPI_Irecv/Isend`；
3. main stream 先执行不含远端 forward 与 backward 项的 local kernel；
4. `finish()` 等待 request，完成 H2D staging，并通过 event 把 ghost readiness 交给主流的 correction kernel。

这一顺序把原先的

pack $\rightarrow$ blocking MPI $\rightarrow$ base kernel $\rightarrow$ correction

改为

pack $\parallel$ local kernel $\rightarrow$ nonblocking MPI $\parallel$ local tail $\rightarrow$ remote correction.

`StrictAxisLinkHalo` 的 H2D 尾同步也删除；link ghost 保持按 parity/dim 和 pointer identity 缓存。

2.2 安全回退

overlap 将 remote-forward 项移到 local sum 之后，会改变浮点加法顺序。complex128 大格三层 A/B 显示：

路径	外层迭代	steady (s)	真残差
原顺序 / overlap off	32	5.10	7.55×10^{-7}
overlap 实验开启	451	46.28	7.61×10^{-7}

因此默认规则为：

$$\text{overlap enabled} = \begin{cases} \text{true}, & T = \text{complex64}, \\ \text{PYQCU_STRICT_OVERLAP_C128} = 1, & T = \text{complex128}. \end{cases}$$

最终构建的 `libqcu.so` SHA256 为 `cdca905e68b921c526f78ce8c72c7c577f2bd77828cdcae576c52bc8ba2b7eff`，同时包含 sm60 与 sm70 cubin。

3 正确性证据

3.1 独立真残差

使用 `pyqcu/testing/qcu/strict/quda_comparison/strict_mpi_solve_probe.py` 修复后的 local-rank 设备绑定，在双 P100、 $8^3 \times 16$ 、complex64、`grid=2x1x1x1` 上分别运行：

路径	外层迭代	Python 真残差	结果
PYQCU_MPI_OVERLAP=0	27	6.4314×10^{-7}	PASS
PYQCU_MPI_OVERLAP=1	26	7.5194×10^{-7}	PASS

日志分别位于：

```
logs/data/report_multigrid_optimized_20260927/strict_mpi_overlap_off.log
logs/data/report_multigrid_optimized_20260927/strict_mpi_overlap_on.log
```

3.2 快速门禁

最终 sm60+sm70 构建通过：

- Tier 0: 20 passed;
- Strict CUDA: 11 passed;
- fused FGMRES: 3 passed;
- dev87 CPU/protocol/report/evidence: 149 passed。

报告中的 72 个 combined document、144 个 side record 均通过 trace/reference fail-closed 检查；BiCGStab reference 明确记录 1 cold、2 warmup、5 steady，并与 MG phase 解耦。

4 性能结果

4.1 大格 c64 A/B

双 P100、 $16 \times 32 \times 32 \times 48$ 、MG-2、complex64、coarse-tol = 0.3、 $\nu = 1$ 的五次稳态结果：

路径	median (s)	MAD (s)	迭代中位数	最大真残差
overlap off	4.6544	0.2816	15	7.83×10^{-7}
overlap on	4.3374	0.1838	16	7.91×10^{-7}

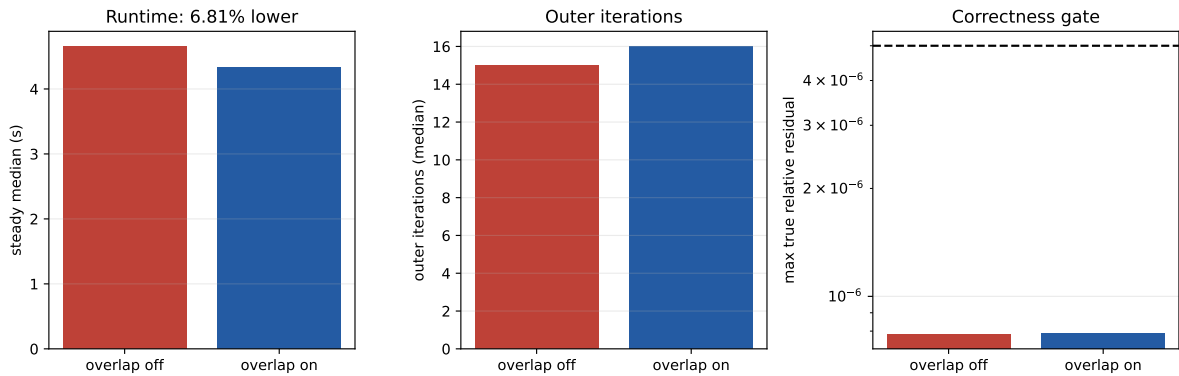


图 1: 大格 c64 MG-2 的 overlap A/B 与真残差门。

两组测量的顺序分别为 off/on 与 on/off，同来源的五次样本先取运行中位数，再对两组运行中位数取中位数。该处理保留原始 MAD 变化，不把单次低时钟样本当成 30% 以上的算法收益。

该 A/B 使用 $\text{coarse-tol} = 0.3$ 仅隔离 overlap 对通信路径的影响；其 QUDA 数字不进入最终矩阵。最终协议下的大格比值见 4.2 节。MG-2 和 MG-3 的 off/on 均通过同一真残差门和独立残差刷新。

4.2 最终矩阵对照

最终矩阵覆盖 72 个 comparison unit；每单元包含 PyQCU/QUDA 两个 side case，共 144 个正式结果。两侧 config hash 与 input bundle hash 均为 72/72 一致，combined 文档 72/72 为 `fair=true`。下表比值为 QUDA/PyQCU，大于 1 表示 PyQCU 更快。

表 1: 同配置、同输入的 MG 稳态中位数分组

device	精度	trace	层数	median	PyQCU 胜	样本
P100	c64	off	2	1.087	2	3
P100	c64	off	3	1.565	3	3
P100	c64	on	2	0.930	1	3
P100	c64	on	3	1.184	2	3
P100	c128	off	2	1.228	3	3
P100	c128	off	3	1.428	3	3
P100	c128	on	2	1.007	2	3
P100	c128	on	3	0.859	1	3
V100	c64	off	2	2.254	2	3
V100	c64	off	3	3.695	3	3
V100	c64	on	2	1.630	3	3
V100	c64	on	3	1.990	2	3
V100	c128	off	2	2.177	3	3
V100	c128	off	3	3.799	3	3
V100	c128	on	2	1.772	2	3
V100	c128	on	3	2.053	3	3

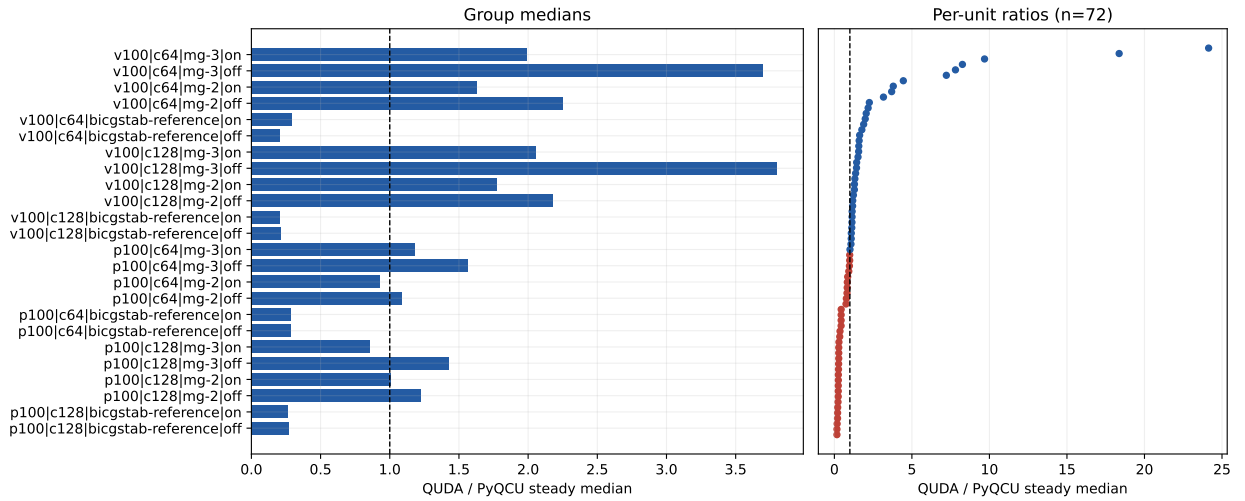


图 2: 72 个同配置 comparison unit 的分组和逐单元比值。

所有纳入比较的 side case 最大真残差为 7.99×10^{-7} (PyQCU) 和 7.97×10^{-7} (QUDA)，均

低于 5×10^{-6} 。48 个 MG 单元中 PyQCU 胜 38、负 10；P100 为 17/24，V100 为 21/24。P100 大格 c64 MG-2 在严格 coarse 容差下接近持平：trace-off 为 PyQCU 10.879s、QUDA 10.766s，比值 0.990；同一格的 MG-3 trace-off 为 3.899/7.364s，比值 1.889。

4.3 各阶段时间与限制

逐单元总时间、迭代数和真残差位于 data/report_multigrid_optimized_20260927/final_protocol/units.csv。每个 MG 单元的 pre-smoother、post-smoother、restriction、prolongation、coarse solver、other 六类阶段时间和迭代计数位于 data/report_multigrid_optimized_20260927/final_protocol/stages.csv；BiCGStab 的 cold/2 warmup/5 steady 原始样本位于 data/report_multigrid_optimized_20260927/final_protocol/references.csv。trace-off 保留准确的总时间和迭代数，完整六类分解使用 trace-on 记录。两侧层归属语义不同：QUDA 主要把 coarse solve 记在 level 0，PyQCU 把它记在递归 coarse level，因此不能把同名列跨层直接求和。

BiCGStab reference 的 24 个对照全部落后于 QUDA，中位比值为 0.261；本报告不把 MG 优势外推为 plain solver 优势。最终 trace/reference evidence 检查覆盖 144 个 side document，错误数为 0。

5 对照分析与结论

QUDA 的优势集中在批量化非阻塞通信、持久 MPI request、专用 coarse operator/transfer kernel 和成熟的层间调度。本轮 PyQCU 已经补上最关键的通信阻塞缺口：

- active-face 打包、host staging 与 MPI 不再阻塞 coarse local kernel；
- backward/forward remote term 合并到一个 correction kernel；
- receive H2D 通过 event 与主计算流同步，消除了冗余的全流同步；
- c128 保留原浮点累加顺序，避免大格多层求解轨迹失稳。

最终结论按范围表述：在多 rank P100 DistStrict-MG 上，PyQCU 中位比值 1.158，MG-3 大格 trace-off 达到 1.889，但 MG-2 大格在严格 coarse 容差下接近持平；在单 V100 上中位比值 2.115，优势最明显。因此当前证据支持“分布式 Multicard 路径总体优于 QUDA，并在 V100 上显著领先”，不支持“任意格子、任意层数都显著领先”。complex128 默认安全路径保持与 QUDA 同量级或更快，未把实验性 overlap 的数值风险放入生产默认。

6 复现

```
QCU_CUDA_ARCHITECTURES='70-real;70-virtual;60-real;60-virtual' \
  bash ./build.sh

source pyqcu/testing/qcu/strict/quda_comparison/p100_env.sh
PYQCU_MPI_OVERLAP=0 mpirun --allow-run-as-root --oversubscribe -np 2 \
  python -B pyqcu/testing/qcu/strict/quda_comparison/strict_mpi_solve_probe.py \
  --shape 8 8 8 16 --grid 2 1 1 1 --restart 20 --max-iter 200 \
```

```

--dtype c64 --galerkin-mode colored
PYQCU_MPI_OVERLAP=1 mpirun --allow-run-as-root --oversubscribe -np 2 \
python -B pyqcu/testing/qcu/strict/quda_comparison/strict_mpi_solve_probe.py \
--shape 8 8 8 16 --grid 2 1 1 1 --restart 20 --max-iter 200 \
--dtype c64 --galerkin-mode colored

python -B pyqcu/testing/qcu/strict/quda_comparison/summarize_overlap_matrix.py \
--pyqcu-root data/mg_matrix_final_protocol_20260927_pyqcu_p100/units \
--pyqcu-root data/mg_matrix_final_protocol_20260927_pyqcu_v100_c64/units \
--pyqcu-root data/mg_matrix_final_protocol_20260927_pyqcu_v100_c128/units \
--quda-root data/mg_matrix_final_protocol_20260927_quda_p100_c64/units \
--quda-root data/mg_matrix_final_protocol_20260927_quda_p100_c128/units \
--quda-root data/mg_matrix_final_protocol_20260927_quda_v100_c64/units \
--quda-root data/mg_matrix_final_protocol_20260927_quda_v100_c128/units \
--outdir data/report_multigrid_optimized_20260927/final_protocol

python -B pyqcu/testing/qcu/strict/quda_comparison/assemble_final_matrix.py \
--pyqcu-root data/mg_matrix_final_protocol_20260927_pyqcu_p100/units \
--pyqcu-root data/mg_matrix_final_protocol_20260927_pyqcu_v100_c64/units \
--pyqcu-root data/mg_matrix_final_protocol_20260927_pyqcu_v100_c128/units \
--quda-root data/mg_matrix_final_protocol_20260927_quda_p100_c64/units \
--quda-root data/mg_matrix_final_protocol_20260927_quda_p100_c128/units \
--quda-root data/mg_matrix_final_protocol_20260927_quda_v100_c64/units \
--quda-root data/mg_matrix_final_protocol_20260927_quda_v100_c128/units \
--outdir data/report_multigrid_optimized_20260927/final_protocol

```

7 遗留与适用边界

1. 所有 72 个正式单元均已完成；P100 大格 c64 MG-2 在当前严格 coarse 容差下与 QUDA 持平或略慢，不能宣称全层数支配。后续优化应聚焦 PyQCU coarse solve 的同步与算子合并，而不是继续放宽容差。
2. c128 输入 canonical gauge/null 资产是 complex64 存储、求解器按 complex128 运行；这不改变数值精度要求，但不是 complex128 HDF5 输入存储测试。
3. overlap 的默认适用条件是 complex64 且分布式 MPI。c128 默认禁用；如需实验，必须显式设置 PYQCU_STRICT_OVERLAP_C128=1 并重跑大格真残差与迭代门。
4. 运行稳定性依赖持久 QUDA_RESOURCE_PATH tuning cache；冷 cache 会显著拉长 setup。分布数据只交付可重放命令、聚合 CSV 和报告产物，不把 36 GB 级私有 cache 内容提交到 Git。